Listening Head Generation

リアルタイム特徴量抽出・レンダリングツール

設計仕様書

バージョン 2.2（最終版）

2026 年 3 月 18 日

MTamon

LHG Tool Design Specification

改訂履歴

版	日付	変更内容	備考
1.0	2026-03-17	初版作成	14 ページ構成
2.0	2026-03-18	レビュー反映（全 16 項目）	ABC 型変更、converters/独立、DECA-FLAME PCA 発見等を反映
2.1	2026-03-18	精読レビュー反映（3 項目）	condition vector 次元統一(120D)、extract_batch 非 abstract 化追記、Phase 分類明記
2.2	2026-03-18	環境要件更新	本文書。Python 3.11.0 / PyTorch 2.9.0 / CUDA 12.8 に更新、シェルスクリプト引用追加

注記: 本文書は v1.0 に対するレビューコメント全 12 項目（回答書 v1）、追加確認 4 項目（回答書 v2）、精読レビュー3 項目（回答書 v3）の全設計決定を統合した最終版です。

目次

改訂履歴 2
目次 3
1. 概要と目的 5
1.1 背景 5
1.2 目的 5
1.3 対象読者 5
2. アーキテクチャ概要 6
2.1 統合パイプライン概要 6
2.2 モジュラー設計の原則 6
2.3 v1.0 からの主要変更点 6
3. ルート A: BFM ベースパイプライン 8
3.1 特徴量抽出: Deep3DFaceRecon 8
3.2 レンダリング: PIRender 8
3.3 互換性と ViCo との関係 8
4. ルート B: FLAME ベースパイプライン 9
4.1 特徴量抽出オプション 9
4.2 レンダリングオプション 9
4.3 L2L（Learning to Listen）との互換性 9
eyes_pose / eyelids の推定方法 10
5. BFM ⇔ FLAME 相互変換 11
5.1 公式ツール: BFM_to_FLAME 11
5.2 FLAME 表現空間の互換性（v2.0 新規調査結果） 11
5.3 実践的推奨 11
6. リアルタイムモード設計 12
6.1 パイプライン 12
6.2 マルチスレッド設計 12
6.3 マルチ GPU 配置戦略 12
6.4 フレームドロップ戦略 12
6.5 速度見積もり 13
7. 前処理（バッチ）モード設計 14
7.1 機能 14
7.2 保存フォーマット 14
7.3 チェックポイント（中断・再開） 14
7.4 CLI インターフェース 14
8. ソフトウェア設計 16
8.1 モジュール構成（v2.0 更新版） 16
8.2 主要インターフェース（Python ABC） 18
BaseExtractor 18
BaseRenderer（v2.0: setup/render 分離） 20
BaseLHGModel（v2.0: 2 引数版） 20

BaseAdapter（v2.0: converters/ に移動）... 22
converters/ モジュールの責務分離... 23
8.3 face_detect.py: 顔検出の責務分離... 24
8.4 buffer.py: パラメータバッファ仕様... 25
8.5 エラーハンドリングポリシー... 26
8.6 ロギングフレームワーク... 26
8.7 設定ファイルスキーマ（YAML）... 27
9. 必要環境・依存関係... 29
9.1 ハードウェア要件... 29
9.2 ソフトウェア要件... 29
9.3 環境構築スクリプト... 29
9.4 モデルファイルサイズ... 33
10. 開発ロードマップ... 34
Phase 1: コア基盤（2-3 週間）... 34
Phase 2: ルート B（FLAME）実装（3-4 週間）← 推奨優先... 35
Phase 3: ルート A（BFM）実装（2-3 週間）... 35
Phase 4: リアルタイムパイプライン（2-3 週間）... 35
Phase 5: 変換ツールとユーティリティ（1-2 週間）... 35
11. 結論と推奨事項... 36
11.1 主推奨: ルート B（FLAME）を主軸とした開発... 36
11.2 ルート A（BFM）の位置づけ... 36
11.3 リアルタイム性能の展望... 36
付録 A: レビューコメント対応一覧... 37

1. 概要と目的

1.1 背景

LHG 研究では、話者の音声・映像から聞き手の自然な頭部動作を生成することを目的とする。この研究パイプラインにおいて、3D Morphable Model（3DMM）のパラメータ抽出と、得られたパラメータからのフォトリアルな顔画像レンダリングは、不可欠な前処理・後処理工程である。

1.2 目的

本ツール（FLARE：**F**acial **L**andmark **A**nalysis & **R**endering **E**ngine）は以下の機能を提供する:

- LHG 研究における特徴量抽出（エンコード）とフォトリアルレンダリング（デコード）の統合ツールを提供する
- 2 つのルート（BFM ベースと FLAME ベース）をサポートし、研究の柔軟性を確保する
- リアルタイムモード（最低 10FPS、理想 30FPS 以上）と前処理モード（バッチ処理）の両方に対応する
- モジュラー設計により、各コンポーネントの独立した交換・拡張を可能にする

1.3 対象読者

- LHG 研究に携わる研究者・エンジニア
- 3DMM 特徴量抽出やフォトリアルレンダリングのツール開発者
- 本ツールを利用して LHG モデルの評価・デモを行うユーザ

2. アーキテクチャ概要

本ツールは、入力から出力まで一貫したパイプラインを提供する統合アーキテクチャを採用する。各モジュールは抽象基底クラス（ABC）を通じてインターフェースが定義されており、独立して交換可能なモジュラー設計となっている。

2.1 統合パイプライン概要

```
[Input Source]
  ├── Webcam / Video Stream (リアルタイムモード)
  └── Dataset Directory (前処理モード)
       ↓
[Face Detection & Crop] ← face_detect.py (共有ユーティリティ)
       ↓
[Feature Extractor Module] ← Route A: Deep3DFaceRecon (BFM)
                                Route B: DECA/SMIRK/SPARK (FLAME)
       ↓
[Parameter Buffer / Storage] ← buffer.py (queue.Queue ベース)
       ↓
[Converter] ← converters/ (DECA→FLAME 等のパラメータ変換)
       ↓
[LHG Model Interface] ← predict(audio_features, speaker_motion)
       ↓
[Converter] ← converters/ (Model 出力→Renderer 入力形式変換)
       ↓
[Renderer Module] ← Route A: PIRender (BFM)
                      Route B: FlashAvatar/HeadGaS (FLAME)
       ↓
[Output]
  ├── Display Window (リアルタイムモード)
  └── Video File / Frame Sequence (バッチモード)
```

2.2 モジュラー設計の原則

- 各コンポーネント（Extractor, Renderer, Model Interface）は抽象基底クラスを継承
- 設定ファイル（YAML）によるコンポーネント選択・パラメータ指定（pydantic バリデーション）
- ルート A（BFM）とルート B（FLAME）のパイプラインを独立して構成可能
- 新しい Extractor や Renderer の追加が、既存コードの変更なしに可能
- パラメータ変換は converters/ モジュールに集約し、AdapterRegistry パターンで自動選択

2.3 v1.0 からの主要変更点

本版では、レビューフィードバック全 16 項目を反映し、以下の重要な設計変更を行った:

- ABC インターフェースの型を np.ndarray から torch.Tensor に統一（GPU 処理完結のため）
- predict() を 2 引数版 (audio_features, speaker_motion) に変更（元の 1 引数版は廃止）

- BaseRenderer に setup()/render() 分離パターンを導入（source_image は初期化時のみ）
- adapter.py を model_interface/ から converters/ に移動し、パラメータ変換専用モジュールとして独立
- DECA exp 50D と FLAME expr 100D が同一 PCA 空間であることを発見 → ゼロパディングで正確に変換可能（線形マッピング不要）
- DECA encode() に内部 FAN 処理がないことを確認 → face_detect.py との共存に改修不要
- YAML config スキーマ、buffer.py 仕様、エラーハンドリング 3 段階、Loguru ロギングを追加定義
- extract_batch() を @abstractmethod からデフォルト実装（ループ版）を持つ通常メソッドに変更（サブクラスでバッチ最適化オーバーライド可能）

3. ルート A: BFM ベースパイプライン

ルート A は、Basel Face Model（BFM）をベースとした特徴量抽出・レンダリングパイプラインである。ViCo Challenge Baseline との互換性を持ち、BFM パラメータ空間での研究に適している。

3.1 特徴量抽出: Deep3DFaceRecon

リポジトリ: sicxu/Deep3DFaceRecon_pytorch

BFM パラメータ構造:

パラメータ	次元数	説明
identity (id)	80	顔形状の個人差を表すパラメータ
expression (exp)	64	表情パラメータ
texture (tex)	80	顔テクスチャパラメータ
pose	6	頭部姿勢（回転 3D + 平行移動 3D）
lighting	27	照明パラメータ（球面調和関数）
合計	257	id 80 + exp 64 + tex 80 + pose 6 + lighting 27

性能特性:

- 推論速度: 約 50+ FPS on GTX 1080（GPU）
- リアルタイム対応: 十分可能
- 必要データ: BFM_model_front.mat（Basel Face Model 公式サイトから登録後ダウンロード）

3.2 レンダリング: PIRender

リポジトリ: RenYurui/PIRender

項目	詳細
入力	BFM パラメータ + ソース画像 1 枚（setup()時に登録）
出力	フォトリアルな顔画像
推論速度	GPU で概ねリアルタイム（~20-30 FPS on modern GPU）
品質	GAN 系、中程度のフォトリアリズム
利点	汎用レンダラー、人物ごとの事前学習不要

3.3 互換性と ViCo との関係

- ViCo Challenge Baseline（dc3ea9f/vico_challenge_baseline）が Deep3DFaceRecon + ViCo + PIRender を統合済み
- L2L のオリジナルは DECA 出力（FLAME 系）だが、BFM で再学習すればルート A で動作可能

注意: L2L の出力は DECA 形式（exp 50 次元 + pose 6 次元 + detail 128 次元 = 184 次元）であり、BFM の exp 64 次元とは別物である。両者のパラメータ空間は直接互換性がないため、混在させないこと。

4. ルート B: FLAME ベースパイプライン

ルート B は、FLAME（Faces Learned with an Articulated Model and Expressions）をベースとした特徴量抽出・レンダリングパイプラインである。最新研究（CVPR 2024/2025）の多くが FLAME ベースであり、L2L との直接互換性を持つため、本ツールの主推奨ルートである。

4.1 特徴量抽出オプション

ツール	推論速度	出力形式	備考
DECA	~8.4ms/image (~119 FPS)	FLAME exp 50 + pose + detail	SIGGRAPH 2021, 最も広く使われている
EMOCA	~11.2ms/image (~89 FPS)	FLAME exp 50 + pose + detail	感情表現に強い
SMIRK	同等速度	FLAME exp 50 + pose	非対称表情に強い、2024 年最新
SPARK	リアルタイム (ResNet-50)	FLAME exp + pose	個人特化、SIGGRAPH Asia 2024
3DDFA V2	~1.35ms/image (~740 FPS)	BFM shape 40 + exp 10	最速だが BFM 系・低次元

推奨: DECA（汎用性と実績）または SMIRK（最新・高品質表情）をデフォルトとし、SPARK をオプションとして提供する。

4.2 レンダリングオプション

ツール	速度	品質	事前学習	GPU 要件
FlashAvatar	300 FPS @ 512x512	極めて高い	RTX 3090 で約 30 分	VRAM 12GB+
HeadGaS	250 FPS @ 512x512	高い	モノキュラー動画から学習	VRAM 12GB+
AGORA	250+ FPS (GPU) / ~9 FPS (CPU)	高い	-	-

推奨: FlashAvatar（300FPS、最高品質、FLAME 直接対応）を主レンダラーとする。

4.3 L2L（Learning to Listen）との互換性

重要な発見（v2.0 新規）: DECA exp 50D と FlashAvatar expr 100D は同一の FLAME expression PCA 空間に属する。両者は同一の generic_model.pkl から shapedirs[:,:,300:300+n_exp] をスライスしており、DECA は n_exp=50（第 1-50 主成分）、FlashAvatar は n_exp=100（第 1-100 主成分）を使用する。

変換方法（ゼロパディング）:

```
# DECA exp (50D) → FlashAvatar expr (100D)
# 同一 PCA 空間のため、ゼロパディングで正確に変換可能
expr_100d = F.pad(deca_exp, (0, 50), value=0.0)  # (B, 50) → (B, 100)

# FlashAvatar condition vector (120D):
#   expr(100D) + jaw_pose(6D rotation) + eyes_pose(12D = 6D rot x2) + eyelids(2D)
```

```
# jaw_pose: DECA pose[:, 3:6] (axis-angle 3D) → rotation_6d (6D) 変換が必要
# eyes_pose: Phase 1 は単位回転行列の 6D 表現 x2、MediaPipe 推定をオプション提供
# eyelids: Phase 1 はゼロ埋め
```

注記: v1.0 で記載していた線形マッピング行列による変換は不要である。ゼロパディングに置き換えることで、事前学習済みモデルや最小二乗法の学習データも不要となり、実装が大幅に簡素化される。

eyes_pose / eyelids の推定方法

方法	精度	実装コスト	説明
A. MediaPipe 推定	高	中	MediaPipe Face Landmarker → FLAME eye pose 変換。mediapipe-blendshapes-to-flame リポジトリ利用可能
B. 単位回転埋め	低〜中	低	eyes_pose は単位回転行列の 6D 表現 x2(12D)、eyelids はゼロ。瞼・視線の微細動作は欠損するが表情全体は正しく表現される
C. 学習ベース推定	高	高	expression/pose から推定する小規模 MLP を学習

Phase 1 では単位回転埋め（方法 B）をデフォルトとし、MediaPipe 推定（方法 A）をオプションとして提供する。

5. BFM ⇔ FLAME 相互変換

5.1 公式ツール: BFM_to_FLAME

リポジトリ: TimoBolkart/BFM_to_FLAME

項目	詳細
機能	BFM メッシュ → FLAME メッシュのトポロジー変換、テクスチャ変換
制限	パラメータ（係数ベクトル）の直接変換ではなく、3D メッシュレベルの変換
対応フォーマット	BFM 2017, BFM 2009, cropped BFM 2009（3DDFA 用）

5.2 FLAME 表現空間の互換性（v2.0 新規調査結果）

ソースコード調査により、以下の重要な事実が判明した:

- DECA と FlashAvatar は共に generic_model.pkl を読み込み、同一の shapedirs テンソルから expression blendshapes を抽出
- DECA: shapedirs[:,:,300:350] → exp 50D（第 1-50 主成分）
- FlashAvatar: shapedirs[:,:,300:400] → expr 100D（第 1-100 主成分）
- 従って DECA exp 50D は FlashAvatar expr 100D の部分空間に完全に含まれる
- 変換は F.pad(exp_50d, (0, 50), value=0.0) のゼロパディングのみで正確

5.3 実践的推奨

- ルート A とルート B を混在させず、どちらかに統一するのが最も実用的
- BFM での評価が必要な場合: BFM_to_FLAME のメッシュ変換を比較評価用に使用
- DECA→FlashAvatar 変換はゼロパディングで対応（converters/deca_to_flame.py）

6. リアルタイムモード設計

6.1 パイプライン

リアルタイムモードでは、Web カメラからの映像入力をリアルタイムに処理し、LHG モデル推論後のレンダリング結果を即座に表示する。

```
Webcam Capture (cv2.VideoCapture)     [~0.5ms]
        ↓
Face Detection & Crop (face_detect.py)  [~3-15ms]
        ↓
3DMM Parameter Extraction (DECA/Deep3DFaceRecon)  [~8-20ms]
        ↓
Parameter Buffer (PipelineBuffer)     [queue.Queue]
        ↓
Converter (converters/)               [パラメータ変換]
        ↓
LHG Model Inference                   [モデル依存]
        ↓
Converter (converters/)               [Renderer 形式変換]
        ↓
Renderer (FlashAvatar/PIRender)       [~3-50ms]
        ↓
Display (cv2.imshow / PyQt)
```

6.2 マルチスレッド設計

スレッド	機能	リソース	デバイス設定
キャプチャスレッド	Web カメラからのフレーム取得	CPU	-
抽出スレッド	顔検出 + 3DMM パラメータ抽出	GPU	device_map.extractor
推論スレッド	LHG モデル推論	GPU	device_map.lhg_model
レンダリングスレッド	画像生成	GPU	device_map.renderer
表示スレッド	GUI 表示	CPU	-

6.3 マルチ GPU 配置戦略

config.yaml の device_map セクションで各コンポーネントのデバイスを指定する。

構成	配置	設定例
1 GPU	全コンポーネント同一デバイス	device_map で全て "cuda:0"
2 GPU	Extractor+LHGModel / Renderer	推論コストの高い Renderer を分離
3 GPU	Extractor / LHGModel / Renderer	完全パイプライン並列

研究段階では単一 GPU 構成がデフォルトで十分であり、マルチ GPU 対応はオプション機能として位置づける。

6.4 フレームドロップ戦略

リアルタイム処理において処理遅延が発生した場合は「最新フレーム優先（Latest-Frame-Wins）」

方式を採用する。

```
class FrameDropPolicy:
    DROP_OLDEST = "drop_oldest"     # デフォルト: 最古フレーム破棄
    BLOCK = "block"                 # バッファ空きまでブロック（バッチ用）
    INTERPOLATE = "interpolate"     # ドロップ後に補間で滑らか化（将来）
```

リアルタイムモードでは DROP_OLDEST をデフォルトとし、バッチモードでは BLOCK を使用する。ドロップ発生時にはログに記録し、統計情報（ドロップ率、最大連続ドロップ数）を収集する。

6.5 速度見積もり

ルート	抽出速度	レンダリング速度	全体見込み
Route A (BFM)	Deep3DFaceRecon ~20ms (~50 FPS)	PIRender ~33-50ms (~20-30 FPS)	20-30 FPS
Route B (FLAME)	DECA ~8.4ms (~119 FPS)	FlashAvatar ~3.3ms (300 FPS)	80-100+ FPS

7. 前処理（バッチ）モード設計

7.1 機能

- データセット内の全動画/フレームを一括処理
- 抽出した特徴量を .npy/.npz 形式でストレージに保存
- 進捗バーとログ出力（Loguru）
- 中断・再開機能（JSON チェックポイント）

7.2 保存フォーマット

```
output_dir/
├── metadata.json        # データセット情報、パラメータ形式、バージョン
├── video_001/
│   ├── params.npz       # shape: (T, param_dim)
│   ├── landmarks.npy    # shape: (T, 68, 2) 2D ランドマーク
│   └── crops.npy        # shape: (T, H, W, 3) クロップ済み顔画像 (オプション)
├── video_002/
│   └── ...
└── summary.csv          # 全動画の統計情報
```

7.3 チェックポイント（中断・再開）

JSON 形式のチェックポイントファイルにより、バッチ処理の中断・再開を実現する。

```
{
  "version": "1.0",
  "created_at": "2026-03-18T10:30:00+09:00",
  "pipeline_config": "config.yaml",
  "input_source": "/data/video_input.mp4",
  "total_frames": 15000,
  "processed_frames": 8421,
  "last_frame_index": 8420,
  "output_dir": "./output/batch_001/",
  "extractor_state": { "last_face_bbox": [120, 80, 380, 340] },
  "metrics": { "avg_fps": 28.5, "dropped_frames": 12, "errors": 0 }
}
```

再開フロー: チェックポイントファイル存在確認 → config の一致検証 → last_frame_index + 1 から処理再開 → 出力ファイルの整合性チェック

チェックポイントは 1000 フレームごと（設定可能）に自動保存される。

7.4 CLI インターフェース

バッチ特徴量抽出:

```
python tool.py extract \
  --input-dir /data/videos/ \
```

```
  --output-dir /data/features/ \
  --route flame --extractor deca \
  --gpu 0 --batch-size 32
```

バッチレンダリング:

```
python tool.py render \
  --input-dir /data/features/ \
  --output-dir /data/rendered/ \
  --route flame --renderer flashavatar \
  --avatar-model /models/avatar_001/ --resolution 512
```

8. ソフトウェア設計

8.1 モジュール構成（v2.0 更新版）

v1.0 から以下の変更を反映: adapter.py を converters/ に移動、converters/registry.py 追加

```
flare/
├── __init__.py
├── config.py                    # 設定管理 (YAML + pydantic)
├── extractors/
│   ├── __init__.py
│   ├── base.py                  # BaseExtractor ABC (Tensor 版)
│   ├── deep3d.py                # Deep3DFaceRecon (BFM)
│   ├── deca.py                  # DECA (FLAME)
│   ├── smirk.py                 # SMIRK (FLAME)
│   └── tdddfa.py               # 3DDFA V2 (BFM, CPU 高速)
├── renderers/
│   ├── __init__.py
│   ├── base.py                  # BaseRenderer ABC (setup/render 分離)
│   ├── pirender.py              # PIRender (BFM)
│   ├── flashavatar.py           # FlashAvatar (FLAME)
│   └── headgas.py               # HeadGaS (FLAME)
├── converters/                  # ← v2.0: 独立モジュール化
│   ├── __init__.py
│   ├── base.py                  # BaseAdapter ABC
│   ├── registry.py              # AdapterRegistry (自動選択)
│   ├── deca_to_flame.py         # DECA→FLAME 変換 (ゼロパディング)
│   ├── flame_to_pirender.py     # FLAME→PIRender 変換
│   └── identity.py              # IdentityAdapter (変換不要時)
├── pipeline/
│   ├── __init__.py
│   ├── realtime.py              # リアルタイムパイプライン
│   ├── batch.py                 # バッチ処理パイプライン
│   └── buffer.py                # PipelineBuffer (queue.Queue)
├── model_interface/
│   ├── __init__.py
│   └── base.py                  # BaseLHGModel ABC (2 引数版)
├── utils/
│   ├── __init__.py
│   ├── video.py                 # 動画 I/O
│   ├── face_detect.py           # 顔検出 (共有ユーティリティ)
│   ├── errors.py                # カスタム例外 + ErrorPolicy
│   ├── logging.py               # Loguru ベースロギング設定
│   ├── visualization.py         # 可視化ユーティリティ
│   └── metrics.py               # 速度計測・ベンチマーク
├── README.md
└── cli.py                       # CLI (argparse/click)
```

8.2 主要インターフェース（Python ABC）

v2.0 変更点: 全インターフェースで torch.Tensor を標準型とし、predict() は 2 引数版、render() は setup()/render() 分離パターンを採用。

BaseExtractor

```
class BaseExtractor(ABC):
    @abstractmethod
    def extract(self, image: torch.Tensor) -> Dict[str, torch.Tensor]:
        """1 フレームから 3DMM パラメータを抽出"""
        pass

    def extract_batch(self, images: torch.Tensor) -> Dict[str, torch.Tensor]:
        """バッチ処理（デフォルト: ループ。サブクラスで最適化可能）"""
        results = {}
        for i in range(images.shape[0]):
            single = self.extract(images[i:i+1])
            for k, v in single.items():
                results.setdefault(k, []).append(v)
        return {k: torch.cat(v, dim=0) for k, v in results.items()}

    @property
    @abstractmethod
    def param_dim(self) -> int:
        """出力パラメータの総次元数"""
        pass

    @property
    @abstractmethod
    def param_keys(self) -> List[str]:
        """出力 Dict のキーリスト"""
        pass
```

extract() 戻り値の Dict キー定義:

Extractor	キー名	次元	説明
DECA	shape	100D	FLAME 形状パラメータ
DECA	tex	50D	テクスチャパラメータ
DECA	exp	50D	表情パラメータ（FLAME PCA 第 1-50 主成分）
DECA	pose	6D	姿勢（global rotation 3D + jaw pose 3D）
DECA	cam	3D	カメラパラメータ
DECA	light	27D	照明（9x3 に reshape）
DECA	detail	128D	詳細パラメータ（E_detail ResNet）
FlashAvatar	shape	300D	FLAME 形状パラメータ
FlashAvatar	expr	100D	FLAME 表情パラメータ（PCA 第 1-100 主成分）
FlashAvatar	jaw_pose	6D	顎回転（6D rotation 表現）

Extractor	キー名	次元	説明
FlashAvatar	eyes_pose	12D	左右眼球回転（6D rotation x2）
FlashAvatar	eyelids	2D	瞼パラメータ

BaseRenderer（v2.0: setup/render 分離）

```
class BaseRenderer(ABC):
    @abstractmethod
    def setup(self, source_image: Optional[torch.Tensor] = None,
              **kwargs) -> None:
        """セッション開始時の初期化。ソース画像の登録等。
        FlashAvatar: NeRF モデルロード
        PIRender: ソース肖像画像の登録"""
        pass

    @abstractmethod
    def render(self, params: Dict[str, torch.Tensor]) -> torch.Tensor:
        """フレームごとのレンダリング。params のみ受け取る。"""
        pass

    @property
    @abstractmethod
    def is_initialized(self) -> bool:
        """setup()済みかどうか"""
        pass
```

設計根拠: PIRender も FlashAvatar もソース画像はセッション開始時に 1 回設定するものであり、フレームごとに変わるものではない。setup() と render() を分離することで、Renderer の種類によらず統一的なパイプライン制御が可能になる。

Renderer	ソース画像の扱い	フレームごとの入力
PIRender	初回 setup() でソース肖像を登録	motion descriptor (BFM exp+pose+trans)
FlashAvatar	不要（学習済み NeRF をロード）	FLAME condition 120D (expr:100 + jaw_pose:6D rot + eyes_pose:12D rot + eyelids:2)

BaseLHGModel（v2.0: 2 引数版）

```
class BaseLHGModel(ABC):
    @abstractmethod
    def predict(
        self,
        audio_features: torch.Tensor,     # (B, T_a, D_a) or (B, D_a)
        speaker_motion: torch.Tensor,     # (B, T_s, D_s) or (B, D_s)
    ) -> torch.Tensor:                    # (B, T_out, D_out) or (B, D_out)
        """音声特徴+話者動作からリスナー動作を予測"""
        pass

    @property
    @abstractmethod
    def requires_window(self) -> bool:
```

```
        """True→ウィンドウレベル入力、False→フレームレベル入力"""
        pass

    @property
    @abstractmethod
    def window_size(self) -> Optional[int]:
        """ウィンドウレベルの場合のフレーム数（例: L2L なら 64）"""
        pass
```

設計根拠: L2L をはじめ主要 LHG モデルは音声特徴量と話者動作を別チャネルで受け取る。元仕様の 1 引数版は廃止し、2 引数版に統一することで責務を明確化する。requires_window プロパティにより、パイプラインがバッファリング戦略を自動切替する。

BaseAdapter（v2.0: converters/ に移動）

```
class BaseAdapter(ABC):
    @abstractmethod
    def convert(self, source_params: Dict[str, torch.Tensor]
               ) -> Dict[str, torch.Tensor]:
        """パラメータ形式を変換する"""
        pass

    @property
    @abstractmethod
    def source_format(self) -> str:  # 例: "deca"
        pass

    @property
    @abstractmethod
    def target_format(self) -> str:  # 例: "flash_avatar"
        pass
```

具体例: DECA → FlashAvatar 変換（ゼロパディング版）：

```
from pytorch3d.transforms import axis_angle_to_matrix, matrix_to_rotation_6d

class DECAToFlameAdapter(BaseAdapter):
    """DECA (50D exp) → FlashAvatar (100D expr) 変換
    同一 FLAME PCA 空間。ゼロパディングで正確に変換。
    jaw_pose: DECA axis-angle(3D) → rotation_6d(6D) に変換。"""

    def convert(self, deca_output: Dict) -> Dict:
        exp_50d = deca_output['exp']           # (B, 50)
        pose = deca_output['pose']              # (B, 6)  global_rot(3) + jaw_aa(3)

        # ゼロパディング: 50D → 100D
        expr_100d = F.pad(exp_50d, (0, 50), value=0.0)  # (B, 100)

        # jaw_pose: axis-angle(3D) → rotation_6d(6D)
        jaw_aa = pose[:, 3:6]                    # (B, 3) axis-angle
        jaw_mat = axis_angle_to_matrix(jaw_aa)   # (B, 3, 3)
        jaw_6d = matrix_to_rotation_6d(jaw_mat)  # (B, 6)

        # eyes_pose: 単位回転行列の 6D 表現 x2 (Phase 1 デフォルト)
        B = exp_50d.shape[0]
        I_6d = matrix_to_rotation_6d(torch.eye(3).unsqueeze(0))  # (1, 6)
        eyes_pose = I_6d.repeat(B, 1).repeat(1, 2)              # (B, 12)

        return {
            'expr': expr_100d,        # (B, 100)
            'jaw_pose': jaw_6d,        # (B, 6) rotation_6d
```

```
            'eyes_pose': eyes_pose,     # (B, 12) identity_6d x2
            'eyelids': torch.zeros(B, 2),      # Phase 1: ゼロ埋め
        }
```

AdapterRegistry パターンにより、config.yaml の extractor.type と renderer.type の組み合わせから適切な Adapter を自動選択する。

converters/ モジュールの責務分離

モジュール	責務	入出力
converters/	パラメータ空間の変換（DECA→FLAME 等）	Dict[str, Tensor] → Dict[str, Tensor]
model_interface/	LHG モデルの ABC 定義と推論	audio + motion → predicted motion
extractors/	画像→3DMM パラメータ抽出	image → Dict[str, Tensor]
renderers/	3DMM パラメータ→画像レンダリング	Dict[str, Tensor] → image

8.3 face_detect.py: 顔検出の責務分離

face_detect.py は共有ユーティリティとして位置づけ、Extractor は検出済みの顔画像のみを受け取る設計とする。

モジュール	責務	詳細
face_detect.py	顔検出・クロッピング	入力画像から顔領域を検出しバウンディングボックスを返す。顔アラインメント（ランドマーク検出含む）
Extractor	3DMM パラメータ推定	face_detect.py で切り出された顔画像を受け取り 3DMM パラメータを推定する（顔検出は行わない）

DECA FAN に関する発見（v2.0）：DECA の encode() 内部に FAN 顔検出は含まれない。FAN は前処理（TestData クラスの datasets/datasets.py）でのみ使用される。したがって、本ツールの face_detect.py と DECA の共存にバイパス改修は不要である。

パイプライン処理フロー:

```
# face_detect.py が顔検出・クロッピングを実行
face_bbox = face_detector.detect(frame)
cropped = face_detector.crop_and_align(frame, face_bbox, size=224)
image_tensor = to_tensor(cropped).to(device)

# Extractor は検出済み画像のみ受け取る（FAN 呼び出しなし）
codedict = deca_model.encode(image_tensor)
```

8.4 buffer.py: パラメータバッファ仕様

Python の queue.Queue ベースの設定可能なバッファ。

パラメータ	型	デフォルト	説明
max_size	int	256	バッファの最大フレーム数
timeout_sec	float	0.5	get() のタイムアウト秒数
overflow_policy	str	drop_oldest	オーバーフロー時の方針

```
class PipelineBuffer:
    def __init__(self, max_size=256, timeout=0.5,
                 overflow_policy="drop_oldest"):
        self._queue = queue.Queue(maxsize=max_size)
        self._timeout = timeout
        self._policy = overflow_policy
        self._stats = {"dropped": 0, "total_put": 0}

    def put(self, frame_data: Dict) -> bool:
        if self._queue.full():
            if self._policy == "drop_oldest":
                self._queue.get_nowait()  # 最古を破棄
                self._stats["dropped"] += 1
            elif self._policy == "block":
                self._queue.put(frame_data)  # ブロック
                return True
        self._queue.put_nowait(frame_data)
        self._stats["total_put"] += 1
        return True

    def get(self) -> Optional[Dict]:
        try:
            return self._queue.get(timeout=self._timeout)
        except queue.Empty:
            return None
```

8.5 エラーハンドリングポリシー

3 段階のエラー処理ポリシーを採用する。リアルタイムモードでは処理継続を最優先とし、バッチモードでは処理精度を優先する。

レベル	対象	動作
SKIP	顔未検出、特徴抽出失敗	当該フレームをスキップし前フレーム結果を保持。ログに WARNING 記録
RETRY	GPU OOM、一時的 I/O エラー	指数バックオフで最大 3 回リトライ。失敗時は SKIP へフォールバック
ABORT	モデルロード失敗、設定不正、致命的エラー	パイプラインを安全に停止。チェックポイント保存後に終了

```
class PipelineErrorHandler:
    def handle(self, error: Exception, context: Dict) -> ErrorPolicy:
        if isinstance(error, FaceNotDetectedError):
            return ErrorPolicy.SKIP
        elif isinstance(error, torch.cuda.OutOfMemoryError):
            torch.cuda.empty_cache()
            return ErrorPolicy.RETRY
        elif isinstance(error, (ModelLoadError, ConfigError)):
            return ErrorPolicy.ABORT
        else:
            logger.error(f"Unexpected error: {error}")
            return ErrorPolicy.SKIP  # リアルタイムでは継続優先
```

8.6 ロギングフレームワーク

Loguru を採用する。研究用ツールにおいて最もシンプルかつ十分な機能を提供する。

フレームワーク	特徴	評価
Loguru（採用）	ゼロ設定、カラー出力、ファイルローテーション、例外キャッチ	研究ツールに最適。pip install loguru のみで即使用可能
structlog	構造化ログ、JSON 出力、コンテキスト変数バインド	プロダクション向け。将来の移行先候補
標準 logging	Python 標準。追加依存なし	設定が煩雑。フォーマッタ・ハンドラの定義が必要

```
from loguru import logger

logger.add(
    "logs/pipeline_{time}.log",
    rotation="10 MB",
    retention="30 days",
    level="INFO",
    format="{time:YYYY-MM-DD HH:mm:ss} | {level} | {module}:{line} | {message}"
)
```

8.7 設定ファイルスキーマ（YAML）

YAML 形式を採用し、pydantic によるバリデーションを行う。

```yaml
# config.yaml
pipeline:
  name: "lhg_realtime_v1"
  fps: 30
  device: "cuda:0"
  converter_chain:
    - type: deca_to_flame
    - type: identity

extractor:
  type: "deca"
  model_path: "./checkpoints/deca_model.tar"
  input_size: 224
  return_keys: ["shape", "exp", "pose", "detail"]

lhg_model:
  type: "learning2listen"
  model_path: "./checkpoints/l2l_vqvae.pth"
  window_size: 64
  codebook_size: 256

renderer:
  type: "flash_avatar"
  model_path: "./checkpoints/flashavatar/"
  source_image: "./data/source_portrait.png"   # setup()時に使用
  output_size: [512, 512]

audio:
  sample_rate: 16000
  feature_type: "mel"     # or "hubert", "wav2vec2"
  n_mels: 128

buffer:
  max_size: 256
  timeout_sec: 0.5
  overflow_policy: "drop_oldest"

device_map:
  extractor: "cuda:0"
  lhg_model: "cuda:0"
  renderer: "cuda:0"

logging:
  level: "INFO"
```

```
  file: "./logs/pipeline.log"
  rotation: "10 MB"

checkpoint:
  enabled: true
  save_dir: "./checkpoints/batch/"
  format: "json"
```

9. 必要環境・依存関係

9.1 ハードウェア要件

項目	最低要件	推奨環境
GPU	RTX 2080 Ti (VRAM 11GB)	RTX 3090 / 4090 (VRAM 24GB)
CPU	Intel i7-8700 / AMD Ryzen 7 3700X	Intel i9-12900K / AMD Ryzen 9 5900X
RAM	16GB	32GB 以上
ストレージ	SSD 100GB 空き	NVMe SSD 500GB 以上

9.2 ソフトウェア要件

コンポーネント	バージョン	備考
Python	3.11.0	pyenv によるバージョン管理
PyTorch	2.9.0 (CUDA 12.8)	torch / torchvision 0.24.0 / torchaudio 2.9.0
pytorch3d	0.7.8（ソースビルド）	axis_angle_to_matrix 等は純 PyTorch 演算のため互換性保証
numpy	2.2.6	数値計算
opencv-python	4.12.0.88	画像・動画処理（opencv-contrib 4.11.0.86）
mediapipe	0.10.11	顔検出（オプション）
pydantic	2.12.4	設定ファイルバリデーション（core 2.41.5）
loguru	0.7+	ロギングフレームワーク
onnxruntime	1.13+	ONNX 推論（オプション）
scipy	1.16.3	科学計算
scikit-learn	1.7.2	機械学習ユーティリティ
librosa	0.11.0	音声特徴量抽出
transformers	4.57.1	HuggingFace モデル（HuBERT / WavLM 等）
accelerate	1.11.0	学習高速化
pytorch-lightning	2.5.6	学習フレームワーク
hydra-core	1.3.2	設定管理（omegaconf 2.3.0）
wandb	0.22.3	実験トラッキング
matplotlib	3.10.7	可視化
pillow	11.3.0	画像処理
huggingface-hub	0.36.0	モデルダウンロード

9.3 環境構築スクリプト

本プロジェクトでは、以下の 2 つのシェルスクリプトにより再現可能な環境構築を行う。

スクリプト	役割・内容
build_environment.sh	CUDA バージョンを引数に取り、install_torch.sh を呼び出した後、約 120 個の pip パッケージを固定バージョンで一括インストール（numpy, scipy, librosa, transformers, hydra-core, wandb 等）
install_torch.sh	PyTorch 2.9.0 + CUDA 12.8 wheel のインストール（torch, torchvision, torchaudio）および関連パッケージ（torchao 0.14.1, torchmetrics 1.8.2, torchsummary 1.5.1, torchtune 0.6.1, torchviz 0.0.3, torchcodec 0.8.1）

注意: pytorch3d はこれらスクリプトに含まれない。PyTorch 2.9.0 環境では公式 wheel が未提供のた

め、ソースからのビルド（git clone + pip install -e .）が必要である。ただし、本ツールで使用する axis_angle_to_matrix / matrix_to_rotation_6d は純 PyTorch テンソル演算であり、C++/CUDA 拡張に依存しないため互換性は保証される。

build_environment.sh

```
# bash build_environment.sh --cuda <CUDA_VERSION>

if [ "$1" != "--cuda" ] || [ -z "$2" ]; then
    echo "Usage: bash install/build_environment.sh --cuda <CUDA_VERSION>"
    exit 1
fi
bash install/python3.11.0/install_torch.sh --cuda $2

pip install --no-deps absl-py==2.3.1
pip install --no-deps accelerate==1.11.0
pip install --no-deps aiohappyeyeballs==2.6.1
pip install --no-deps aiohttp==3.13.2
pip install --no-deps aiosignal==1.4.0
pip install --no-deps annotated-types==0.7.0
pip install --no-deps antlr4-python3-runtime==4.9.3
pip install --no-deps anyio==4.11.0
pip install --no-deps attrs==25.4.0
pip install --no-deps audioread==3.1.0
pip install --no-deps blobfile==3.1.0
pip install --no-deps certifi==2025.10.5
pip install --no-deps cffi==2.0.0
pip install --no-deps charset-normalizer==3.4.4
pip install --no-deps click==8.3.0
pip install --no-deps cmpfilter==0.0.1
pip install --no-deps contourpy==1.3.3
pip install --no-deps cycler==0.12.1
pip install --no-deps Cython==0.29.35
pip install --no-deps datasets==4.4.1
pip install --no-deps decorator==5.2.1
pip install --no-deps dfcon==0.3.0
pip install --no-deps dill==0.4.0
pip install --no-deps dtaidistance==2.3.12
pip install --no-deps easy-video==0.0.4
pip install --no-deps evaluate==0.4.6
pip install --no-deps filelock==3.19.1
pip install --no-deps flatbuffers==25.9.23
pip install --no-deps fonttools==4.60.1
pip install --no-deps frozenlist==1.8.0
pip install --no-deps gitdb==4.0.12
pip install --no-deps GitPython==3.1.45
pip install --no-deps graphviz==0.21
pip install --no-deps h11==0.16.0
pip install --no-deps hf-xet==1.2.0
```

```
pip install --no-deps httpcore==1.0.9
pip install --no-deps httpx==0.28.1
pip install --no-deps huggingface-hub==0.36.0
pip install --no-deps hydra-core==1.3.2
pip install --no-deps idna==3.11
pip install --no-deps ImageIO==2.37.2
pip install --no-deps imageio-ffmpeg==0.6.0
# pip install --no-deps importlib_metadata==8.0.0 # unnecessary?
# pip install --no-deps importlib_resources==6.4.0 # unnecessary?
pip install --no-deps jax==0.7.1
pip install --no-deps jaxlib==0.7.1
pip install --no-deps jiwer==4.0.0
pip install --no-deps joblib==1.3.2
pip install --no-deps kagglehub==0.3.13
pip install --no-deps kiwisolver==1.4.9
pip install --no-deps lazy_loader==0.4
pip install --no-deps librosa==0.11.0
pip install --no-deps lightning-utilities==0.15.2
pip install --no-deps llvmlite==0.45.1
pip install --no-deps lxml==6.0.2
pip install --no-deps matplotlib==3.10.7
pip install --no-deps mediapipe==0.10.11
pip install --no-deps ml-dtypes==0.5.3
pip install --no-deps moviepy==2.2.1
pip install --no-deps msgpack==1.1.2
pip install --no-deps multidict==6.7.0
pip install --no-deps multiprocess==0.70.18
pip install --no-deps networkx==3.5
pip install --no-deps numba==0.62.1
pip install --no-deps numpy==2.2.6
pip install --no-deps omegaconf==2.3.0
pip install --no-deps opencv-contrib-python==4.11.0.86
pip install --no-deps opencv-python==4.12.0.88
pip install --no-deps opt_einsum==3.4.0
pip install --no-deps packaging==25.0
pip install --no-deps pandas==2.3.3
pip install --no-deps pillow==11.3.0
pip install --no-deps platformdirs==4.5.0
pip install --no-deps pooch==1.8.2
pip install --no-deps proglog==0.1.12
pip install --no-deps propcache==0.4.1
pip install --no-deps protobuf==3.20.3
pip install --no-deps psutil==7.1.3
pip install --no-deps pyarrow==22.0.0
pip install --no-deps pycparser==2.23
pip install --no-deps pycryptodomex==3.23.0
pip install --no-deps pydantic==2.12.4
pip install --no-deps pydantic_core==2.41.5
```

```
pip install --no-deps pyparsing==3.2.5
pip install --no-deps python-dateutil==2.9.0.post0
pip install --no-deps python-dotenv==1.2.1
pip install --no-deps pytorch-lightning==2.5.6
pip install --no-deps pytz==2025.2
pip install --no-deps pyworld==0.3.5
pip install --no-deps PyYAML==6.0.3
pip install --no-deps rapidfuzz==3.14.3
pip install --no-deps regex==2025.11.3
pip install --no-deps requests==2.32.5
pip install --no-deps safetensors==0.6.2
pip install --no-deps schedulefree==1.4.1
pip install --no-deps scikit-learn==1.7.2
pip install --no-deps scipy==1.16.3
pip install --no-deps sentencepiece==0.2.1
pip install --no-deps sentry-sdk==2.43.0
pip install --no-deps shellingham==1.5.4
pip install --no-deps six==1.17.0
pip install --no-deps smmap==5.0.2
pip install --no-deps sniffio==1.3.1
pip install --no-deps sounddevice==0.5.3
pip install --no-deps soundfile==0.13.1
pip install --no-deps soxr==1.0.0
pip install --no-deps sympy==1.14.0
pip install --no-deps threadpoolctl==3.6.0
pip install --no-deps tiktoken==0.12.0
pip install --no-deps tokenizers==0.22.1
pip install --no-deps toolpack==0.0.5
pip install --no-deps tqdm==4.66.3
pip install --no-deps transformers==4.57.1
pip install --no-deps typer-slim==0.20.0
pip install --no-deps typing-inspection==0.4.2
pip install --no-deps typing_extensions==4.15.0
pip install --no-deps tzdata==2025.2
pip install --no-deps urllib3==2.5.0
pip install --no-deps wandb==0.22.3
pip install --no-deps xxhash==3.6.0
pip install --no-deps yarl==1.22.0
pip install --no-deps zipp==3.23.0
```

Install_torch.sh

```
# bash install_torch.sh --cuda 11.1

if [ "$1" != "--cuda" ] || [ -z "$2" ]; then
    echo "Usage: bash install_torch.sh --cuda <CUDA_VERSION>"
```

```
    exit 1
fi

CUDA_VERSION=$2 # 12.8
if [ "$CUDA_VERSION" = "12.8" ]; then
    # torch==2.9.0+cu128 などがこれで install できる
    pip install torch==2.9.0 torchvision==0.24.0 torchaudio==2.9.0 --index-url
https://download.pytorch.org/whl/cu128
else
    echo "Usage: bash install_torch.sh --cuda <CUDA_VERSION>"
    echo "CUDA_VERSION must be 12.8"
    exit 1
fi

pip install --no-deps torchao==0.14.1
# pip install --no-deps torchao==0.3.1 # this is not available due to the
import error
pip install --no-deps torchmetrics==1.8.2
pip install --no-deps torchsummary==1.5.1
pip install --no-deps torchtune==0.6.1
pip install --no-deps torchviz==0.0.3
pip install --no-deps torchcodec==0.8.1
```

9.4 モデルファイルサイズ

- BFM モデルファイル: ~100MB
- FLAME モデルファイル: ~50MB (generic_model.pkl 含む)
- 事前学習済みモデル: ~500MB-1GB
- FlashAvatar 使用時: 追加で diff-gaussian-rasterization パッケージが必要

10. 開発ロードマップ

合計 10-15 週間の開発期間を想定する。

Phase 1: コア基盤（2-3 週間）

Phase 1 ファイル生成順序（v2.0 更新版、全 25 ファイル）：

Step 1: パッケージ骨格

1. `flare/__init__.py`
2. `flare/config.py`

Step 2: ABC 定義 + Converter 基盤

3. `flare/extractors/base.py (BaseExtractor: Tensor 版)`
4. `flare/renderers/base.py (BaseRenderer: setup/render 分離)`
5. `flare/model_interface/base.py (BaseLHGModel: 2 引数版)`
6. `flare/converters/base.py (BaseAdapter ABC)`
7. `flare/converters/registry.py (AdapterRegistry)`

Step 3: エラー処理・ロギング基盤

8. `flare/utils/errors.py (ErrorPolicy + カスタム例外)`
9. `flare/utils/logging.py (Loguru ベース)`

Step 4: ユーティリティ

10. `flare/utils/video.py`
11. `flare/utils/face_detect.py`
12. `flare/utils/metrics.py`
13. `flare/utils/visualization.py`

Step 5: パイプライン骨格

14. `flare/pipeline/buffer.py (PipelineBuffer)`
15. `flare/pipeline/batch.py`
16. `flare/pipeline/realtime.py`

Step 6: CLI

17. `flare/cli.py`

Step 7: テスト

18. `tests/conftest.py`
19. `tests/test_config.py`
20. `tests/test_extractors_base.py`
21. `tests/test_renderers_base.py`
22. `tests/test_model_base.py`
23. `tests/test_converter.py (DECAToFlame ゼロパディング検証)`
24. `tests/test_buffer.py`
25. `tests/test_error_handler.py`

Phase 2: ルート B（FLAME）実装（3-4 週間）← 推奨優先

- DECA/SMIRK Extractor の実装
- FlashAvatar Renderer の実装（setup/render 分離パターン）
- converters/deca_to_flame.py（ゼロパディング変換）の実装
- converters/identity.py（パラメータ形式共通時のパススルー変換）
- バッチ処理パイプライン
- L2L 出力との互換性検証

Phase 3: ルート A（BFM）実装（2-3 週間）

- Deep3DFaceRecon Extractor の実装
- PIRender Renderer の実装
- converters/flame_to_pirender.py（FLAME→PIRender 形式変換）
- ViCo Baseline との互換性検証

Phase 4: リアルタイムパイプライン（2-3 週間）

- マルチスレッドパイプライン構築
- Webcam キャプチャと GUI（PyQt/OpenCV）
- レイテンシ最適化
- フレームドロップ戦略（FrameDropPolicy）の実装

Phase 5: 変換ツールとユーティリティ（1-2 週間）

- BFM_to_FLAME メッシュ変換統合
- MediaPipe eyes_pose/eyelids 推定（オプション）
- ベンチマーキングツール
- ドキュメント整備

11. 結論と推奨事項

11.1 主推奨: ルート B（FLAME）を主軸とした開発

ルート B（FLAME）を主軸に開発することを強く推奨する:

- L2L の DECA 出力との完全互換: 同一 PCA 空間のためゼロパディングで正確に変換可能
- FlashAvatar の圧倒的なレンダリング速度（300 FPS）: リアルタイム要件を大幅に上回る
- FLAME-Universe の豊富なエコシステム: DECA, EMOCA, SMIRK, SPARK など多数の Extractor
- 最新研究（CVPR 2024/2025）の多くが FLAME ベース: 将来の拡張性が高い

11.2 ルート A（BFM）の位置づけ

- ViCo 評価互換性のために並行サポートする
- BFM ベースの既存研究資産の活用に有用
- 開発優先度はルート B より低い

11.3 リアルタイム性能の展望

ルート B（FLAME）を採用した場合、抽出（DECA: ~119 FPS）とレンダリング（FlashAvatar: 300 FPS）の両方がリアルタイム要件を大幅に上回る。LHG モデル自体の推論速度がボトルネックとなる可能性が高いが、パイプライン並列化により 30FPS 以上の目標は十分に達成可能と見込まれる。

付録 A: レビューコメント対応一覧

以下に、全 19 項目のレビューコメントとその設計決定、本仕様書での反映箇所を示す。

回答書 v1（全 12 項目）：

#	コメント	設計決定	反映箇所
1	extract() Dict keys 未定義	BFM/FLAME 別キー定義。Tensor 統一	8.2 BaseExtractor
2	render() params 構造不明	Renderer 依存。adapter.py で変換	8.2 BaseRenderer
3	predict() 入力形状未定義	requires_window で柔軟切替。2 引数版	8.2 BaseLHGModel
4	Config スキーマ未定義	YAML + pydantic バリデーション	8.7 Config
5	マルチ GPU 配置未定義	device_map で 1-3 GPU 対応	6.3
6	フレームドロップ戦略未定義	Latest-Frame-Wins, 3 ポリシー	6.4
7	バッチ再開メカニズム未定義	JSON チェックポイント	7.3
8	buffer.py 仕様未定義	queue.Queue, max_size=256	8.4
9	adapter.py 具体例不足	L2L→FlashAvatar 変換例。Registry	8.2 BaseAdapter
10	face_detect 役割分離不明確	共有ユーティリティとして独立	8.3
11	エラー処理方針未定義	3 段階 SKIP/RETRY/ABORT	8.5
12	ロギング未選定	Loguru 推奨	8.6

回答書 v2（追加 4 項目）：

#	確認事項	設計決定	反映箇所
13	ABC 型: np.ndarray vs Tensor	全て torch.Tensor に統一	8.2 全 ABC
14	predict() 1 引数 vs 2 引数	2 引数版採用、1 引数廃止	8.2 BaseLHGModel
15	render() source_image 扱い	setup()/render() 分離	8.2 BaseRenderer
16	adapter.py の配置場所	converters/ に移動・独立	8.1, 8.2

追加発見事項:

#	発見事項	影響	反映箇所
-	DECA/FLAME 同一 PCA 空間	ゼロパディング変換。線形マッピング不要	4.3, 5.2, 8.2
-	DECA encode()に FAN 処理なし	face_detect.py 共存に改修不要	8.3
-	FlashAvatar condition 120D 構成	expr(100)+jaw(6)+eyes(12)+eyelids(2)	4.3, 8.2

回答書 v3（精読レビュー 3 項目）：

#	指摘事項	設計決定	反映箇所
17	condition vector 次元不整合 (jaw 3D→6D, eyes 6→12)	jaw: axis-angle→6D rotation, eyes: identity_6d x2 (12D)	4.3, 8.2, Adapter コード
18	extract_batch() 非 abstract	2.3 の主要変更点に追加明記	2.3

#	指摘事項	設計決定	反映箇所
	化が未記載		
19	identity.py / flame_to_pirender.py の Phase 分類欠如	Phase 2 / Phase 3 にそれぞれ追加	10. ロードマップ

— 以上 —